UP1: FICHEROS

TAREA 1 - PREPARACIÓN DEL ENTORNO Y ESQUELETO DEL PROYECTO

Durante esta unidad desarrollaremos una aplicación Java para gestionar la ludoteca del IES. No construiremos toda la aplicación de una vez: partiremos de una versión sencilla y añadiremos nuevas funcionalidades conforme avancemos en la unidad y vayáis adquiriendo nuevos conocimientos.

En esta primera tarea prepararemos el entorno de trabajo, configuraremos Git y crearemos las clases Videojuego y Main.

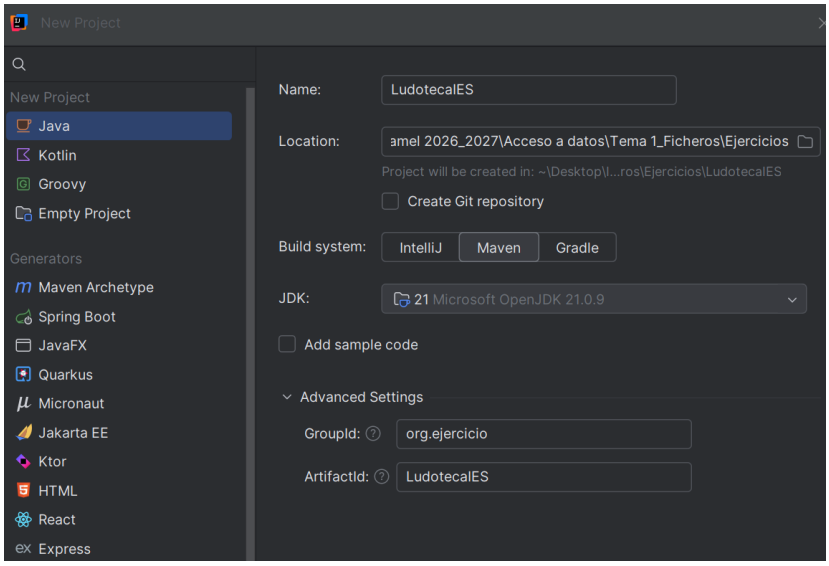
> *El código debe escribirse sin asistentes de inteligencia artificial y sin aceptar sugerencias automáticas de líneas o bloques de código. Si se detecta alguno de estos usos, la tarea será anulada suponiendo un 0 en esta entrega.*

OBJETIVOS

- Configurar IntelliJ IDEA para trabajar con Java.
- Desactivar los asistentes de inteligencia artificial y las sugerencias automáticas de código.
- Crear las clases Videojuego y Main y ejecutar la primera versión.
- Inicializar un repositorio Git y realizar el primer commit.
- Preparar el entorno de test para trabajar con JUnit

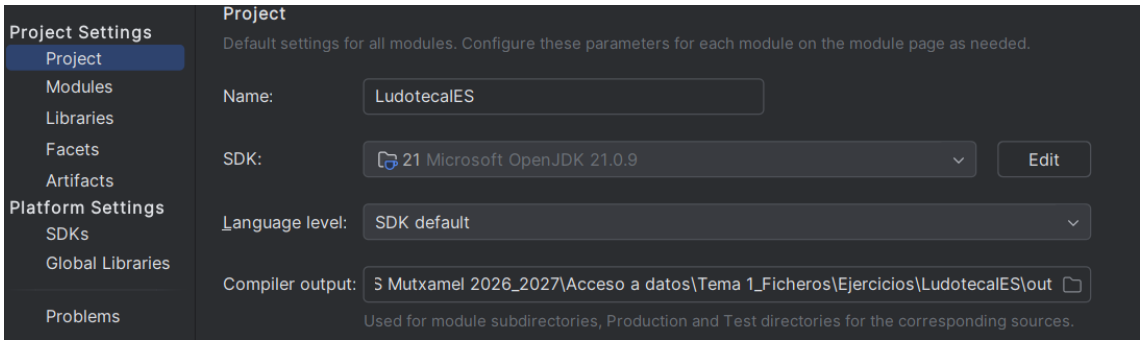
1 PREPARAR INTELLIJ IDEA

1.1Crear el Proyecto "LudotecaIES"



- **Create Git repository**: desmarcado, porque lo inicializaréis manualmente con git init.
- **Add sample code: desmárcalo**, para que creen ellos Main desde cero.

Después de crear el proyecto, comprueba el JDK en el menú *File -> Project Structure -> Project*.

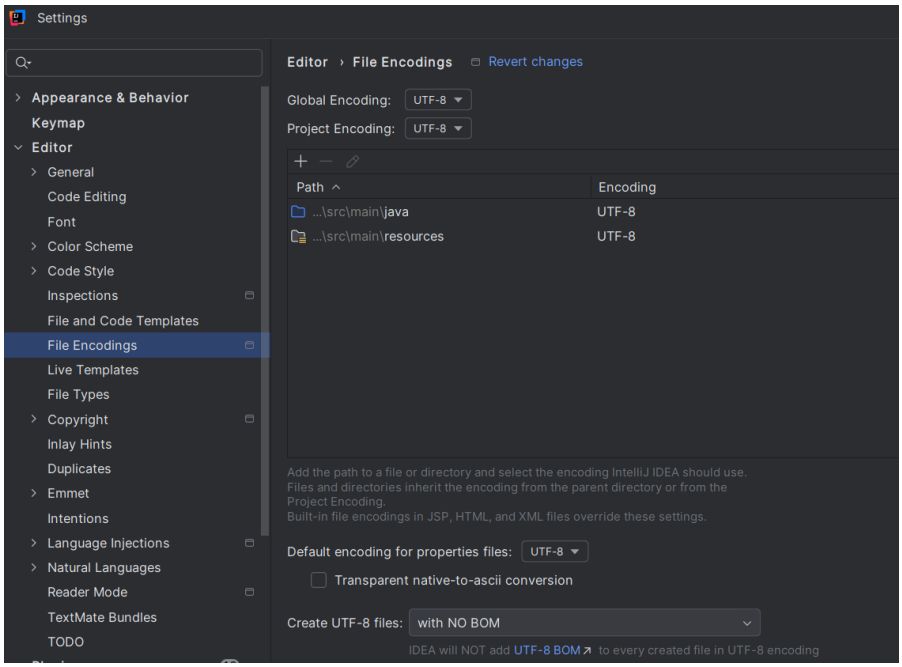


1.2. Configurar UTF-8

Abre el menú File -> Settings -> Editor -> File Encodings y selecciona UTF-8 en:

- Global Encoding.
- Project Encoding.
- Default encoding for properties files.

Esta configuración permitirá conservar correctamente títulos como Órbita y Crónicas del Bosque.



1.3 Desactivar la inteligencia artificial

1. Abre el menú *File -> Settings* o utiliza Ctrl + Alt + S.
2. Entra en Plugins -> Installed y busca AI Assistant (y todo lo relacionado con IA).
3. Si aparece instalado, abre sus opciones, selecciona Disable y reinicia IntelliJ si lo solicita.
4. Busca también GitHub Copilot, Tabnine o Codeium y desactívalos si aparecen instalados.

1.4 Desactivar las sugerencias automáticas

Primero desactiva las propuestas generadas por modelos locales:

`Settings -> Editor -> General -> Code Completion -> Inline`

Desmarcar: `Enable inline completion using language models`

Después desactiva la ventana automática de autocompletado:

`Settings -> Editor -> General -> Code Completion -> Popup`

Desmarcar: `Show suggestions as you type`

Desmarcar: `Insert selected suggestion by pressing space, dot, or other context-dependent keys`

La combinación Ctrl + Space puede seguir mostrando manualmente la ayuda de completado, por lo que no debe utilizarse durante esta actividad. Si una opción no aparece en esa ubicación, búscala dentro de Settings escribiendo inline completion o show suggestions as you type.

2. Preparar Git

Git es un sistema de control de versiones que permite guardar distintos estados del proyecto y consultar los cambios realizados durante su desarrollo.

Con estas primeras tareas trabajaremos únicamente con un repositorio local. Esto significa que tanto el proyecto como su historial estarán almacenados en el ordenador. Más adelante conectaremos el repositorio con GitHub para disponer de una copia remota y compartir el proyecto.

Para realizar esta parte debes seguir el documento en PDF que se encuentra en Aules llamado: **Guía de Git desde IntelliJ IDEA**

Sigue la guía en el orden indicado. No es necesario buscar otros manuales ni repetir procedimientos diferentes.

Este punto es una tarea entregable por lo que, sigue los pasos indicaos y plásmalo en un documento de Word que deberás subir a Aules en formato PDF.

3 CREAR LA PRIMERA VERSIÓN

3.1 Crear el paquete

En el panel Project, pulsa con el botón derecho sobre `src/main/java` y selecciona:

`New -> Package`

Escribe como nombre del paquete: `es.iesmutxamel.ludoteca`

Todas las clases de la actividad deben crearse dentro de este paquete.

3.2 Crear la clase Videojuego

Crea una clase Videojuego con estos atributos privados:

```
private int id;
private String titulo;
private String plataforma;
private double precio;
private int stock;
```

La clase debe incluir:

- Un constructor que reciba todos los atributos.
- Los métodos get necesarios.

- Un método toString() que muestre la información completa de forma legible.

En esta primera sesión no es necesario crear métodos set ni implementar todas las validaciones.

3.3 Crear la clase Main

Crea una clase Main con el método principal de la aplicación:

```
public static void main(String[] args) {


}
```

Dentro de main:

- Crea un Videojuego con ID 101, título Órbita, plataforma PC, precio 19.95 y stock 7.
- Crea un segundo videojuego con datos elegidos por ti.
- Muestra ambos objetos mediante System.out.println().
- La salida debe ser legible. Por ejemplo:

```
ID: 101 | Órbita | PC | 19.95 € | Stock: 7
```

4 Probar y registrar el trabajo

4.1 Comprobar el programa

- El proyecto compila sin errores.
- Se muestran los dos videojuegos.
- Los textos con tilde se visualizan correctamente.

4.2 Realizar el primer commit

- Comprueba primero los archivos pendientes:

  ```
  git status
  ```

- Añade los archivos y registra la primera versión:

  git add .

  git commit -m "Tarea 1: crear primera versión de la ludoteca"

- Comprueba que el commit existe:

  git log –oneline

5 Entrega

- Código fuente del proyecto.
- Captura de la consola mostrando los dos videojuegos.

6 Próxima tarea y JUnit

En la siguiente tarea crearemos la clase Catalogo, almacenaremos varios videojuegos en un ArrayList, añadiremos un menú y comprobaremos qué sucede con los datos al cerrar la aplicación.

Utilizaremos JUnit para automatizar algunas comprobaciones. En esta sesión no hay que crear pruebas unitarias.